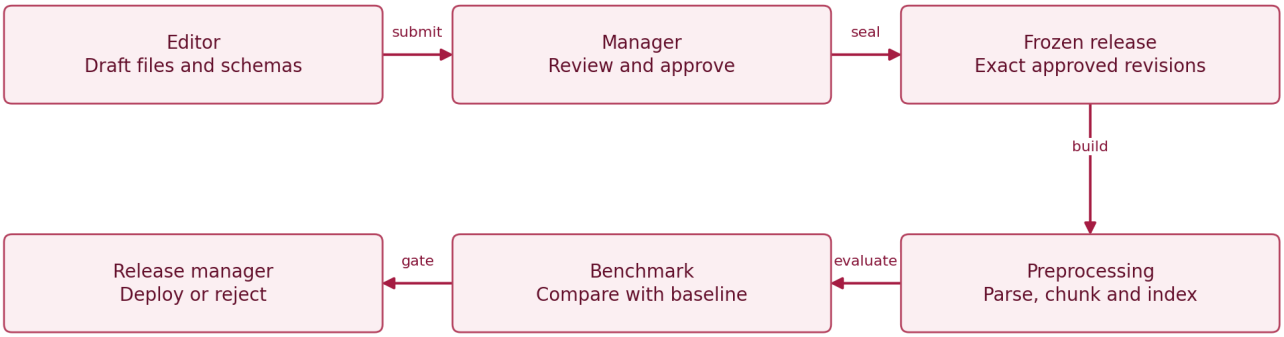
RAG knowledge base management system

Engineering specification • Revision 0.3 • 12 October 2026

Build a dashboard for governed document editing, parsing and release management. Every deployed knowledge base must be traceable to source revisions, parsing rules, responsible users and benchmark evidence. Defaults below are proposed for engineering review.



Role	Allowed actions	Limits
Viewer	Read permitted files, releases and results.	No modification.
Editor	Add files, edit drafts, propose schemas and submit.	Cannot approve own work or deploy.
Manager	Review changes, approve and assemble releases.	Independent reviewer required.
Release manager	Deploy or roll back eligible builds.	Cannot alter approved artifacts.
Admin / auditor	Admin manages grants; auditor reads history.	Neither role implies content approval.

Use one login with server-side role checks. Scope grants by organization → workspace → knowledge base; document restrictions narrow inherited access. Separate agent-user retrieval entitlements from dashboard roles. Workers receive narrowly scoped service identities.

MVP inputs: TXT, Markdown, DOCX and XLSX. Edit plain text in the browser; preserve Office originals and upload edited replacements. In-browser Word/Excel editing is a separate scope decision. Preview text, tables and chunks before submission.

Save with an expected base revision to detect concurrent edits. Submission freezes revision IDs; a subsequent edit needs new approval. Show text/paragraph diffs and Excel row/cell changes, matched by business keys when available. Never silently overwrite a conflicting draft.

2 Versioning and change attribution

Version content, interpretation, derived builds and evaluation separately. Human labels such as KB 1.4 are display names; immutable IDs and manifest hashes establish exact identity.



Record	Required pinned information
Document revision	Document ID, parent revision, blob hash, metadata, author and change reason.
Schema revision	Validated definition, schema-language version, config hash and approval.
KB release	Exact document revisions and schema bindings, parent release and approval evidence.
Build	Release, parser/chunker/tokenizer versions, embedding settings, executable digest and artifact hashes.
Benchmark run	Build, dataset revision, retriever/reranker/generator, prompt, evaluator and raw results.
Deployment	Environment, build, gate evidence, previous deployment, actor and activation time.

A schema-only change creates a new release. A parser or embedding change creates a new build. A prompt or evaluator change creates a new experiment. Resolve all references before processing; never let a running job follow a mutable latest pointer.

For every change, record who, what, when, why, previous/next revision, approver and deployment. Use stable identity-provider user IDs plus display-name snapshots. Commit the mutation and audit event together; preserve attribution after account deactivation.

Deleting content removes it from a successor release. Historical bytes follow retention policy. Rollback creates a new deployment pointing to a retained healthy build. Current access restrictions still apply to old releases, previews, exports and retrieval.

A price cell changes in workbook revision 8 → KB release 12 selects revision 8 and schema 3 → build 21 creates new chunks → benchmark run 44 compares against build 20 → a deployment records build 21 and its reviewer. Each link is navigable in the dashboard.

3 Parsing schemas and preprocessing

Maintain a versioned schema registry with a form editor, validated JSON, sample parse preview and independent approval. Explicitly bind each file to a schema revision. Missing required structures fail validation; generic fallback must never hide schema drift.

Input	Schema must define	Chunking and provenance
Text	Encoding, language, heading boundaries and normalization.	Split by paragraph/section; retain heading path and source offsets.
Word	Included styles, heading hierarchy, table selectors, merged cells, notes and tracked-change policy.	Keep captions and column labels; retain paragraph IDs and table/cell coordinates.
Excel	Sheets/ranges, header rows, column types, business keys, units, nulls, hidden rows and formula policy.	Use rows or coherent groups; repeat headers and units; retain sheet, cells and row keys.

input: XLSX sheet: Products
header_rows: [1] primary_key: [product_id]
required_columns: [product_id, category, description, price]
types: {product_id: string, price: decimal}
formula_policy: cached_value_with_provenance
hidden_rows: exclude chunk_strategy: row_group
group_by: [category] max_tokens: 600
overlap_tokens: 0 repeat_column_headers: true
missing_required_column: fail; duplicate_primary_key: fail

This is a proposed application contract. Token limits require benchmark tuning. Never execute macros or external links. Keep formula text as provenance; missing required cached values fail, while potentially stale caches trigger a visible freshness warning. Do not invent merged-cell values or silently discard unsupported objects.

Boundary	Contract
Input	Job/attempt ID, tenant, frozen release/hash, schema snapshots, pinned pipeline configuration and scoped source references.
Stages	Quarantine and validate → extract → normalize → chunk → embed → build isolated index → verify and seal.
Output	Artifact manifest and hashes, source-linked elements/chunks, counts, warnings, errors and execution metadata.
Failure handling	Idempotent requests, bounded transient retries, worker leases and fenced completion. Schema errors require correction; failed builds cannot publish.

Each chunk stores build, source revision, schema revision, content hash, token count, locator and access-policy reference. Reuse artifacts only when all relevant inputs/configurations match. Switch the active build atomically after gates pass; each agent request uses one pinned build.

4 Architecture evaluation and delivery

Use a modular API backend, PostgreSQL for metadata, object storage for originals/artifacts, a durable queue for workers, and an adapter to the existing vector engine and agent. Separate heavy parsing from interactive requests. Final framework choices depend on the existing stack.

API capability	Core contract
Revisions and schemas	Create immutable revisions; reject stale base versions; authorize both sides of a diff.
Review and releases	Submit/approve pinned change digests; seal exact manifest membership.
Builds and evaluations	Asynchronous jobs return a job ID; requests use idempotency keys and pinned configurations.
Deployment and audit	Compare-and-swap environment pointer; record actor, evidence and prior build; expose filtered history.

Measure	Dashboard illustration and rule
Retrieval	Recall@k, MRR@k and nDCG@k; baseline/candidate bars and slice heatmaps. Freeze relevance units and k.
Answers	Correctness, groundedness, citation support and abstention; per-question drilldown with rubric/judge version.
Performance	p50/p95 latency, token use and cost; distributions and cost-versus-quality scatterplots.
Ingestion and security	Parse failures, omitted rows, chunk sizes and unauthorized retrievals; source-linked diagnostics.

Persist raw results and plot specifications with run IDs, dataset revision, sample counts and exclusions. Compare matching configurations to attribute changes to content; flag confounded comparisons. Keep failed cases visible. Use repeated runs and paired uncertainty estimates where appropriate.

Require a complete build, independent approval, required benchmark coverage, zero unauthorized retrievals on the security suite, and agreed quality/latency limits. Establish numeric thresholds from a baseline. Any override needs explicit authority and an audit reason.

Apply access checks to APIs, files, previews, chunks and results. Validate uploads and sandbox parsers [1]; use a non-owner database role and test row-security policies [2]. Keep audit writes restricted, back up metadata and artifacts, and test restoration. Correlate job traces, metrics and logs [3].

Implement in order: identity/history → schema preview and review → immutable builds/deployment → evaluation/plots. Acceptance must demonstrate conflict detection, Excel/Word edge cases, end-to-end provenance, duplicate-job safety, no partial deployment, access revocation and rollback.

Confirm before sizing: in-browser Office editing or replacement upload; existing preprocessing/agent contracts; access scopes; corpus size and languages; benchmark ownership and retention.

References: [1] OWASP File Upload Cheat Sheet, cheatsheetseries.owasp.org. [2] PostgreSQL Row Security Policies, postgresql.org/docs/current/ddl-rowsecurity.html. [3] OpenTelemetry Signals, opentelemetry.io/docs/concepts/signals/.